

Classificação do Nível de Stress

Machine Learning 2026 – LIACD/LECD/LEEC

Departamento de Engenharia Informática

Duarte Prazeres

Abril 2026

Índice

1 Introdução **3**

2 Arquitetura e Modularidade do Projeto **3**

3 Dataset e Análise Exploratória **3**
3.1 Descrição do Dataset 3
3.2 Limpeza e Qualidade dos Dados 4
3.3 Visualizações EDA 4

4 Transformação do Target **6**

5 Feature Engineering **6**

6 Pré-processamento **7**
6.1 Divisão dos Dados 7
6.2 Normalização e Prevenção de Data Leakage 8
6.3 Lidar com o Desequilíbrio de Classes 8

7 Métricas de Avaliação **8**

8 Modelos Implementados **9**
8.1 Support Vector Machine (SVM) 9
8.2 Rede Neuronal (MLPClassifier) 10
8.3 XGBoost – 3.º Modelo 10

9 Validação Cruzada (Cross-Validation) **11**

10 Pipeline de Produção e Guardar Modelos (.pkl) **12**

11 Resultados Finais de Avaliação **12**
11.1 Confusion Matrices – Cenário A 13
11.2 Confusion Matrices – Cenário B 14
11.3 Confusion Matrices – Cenário C 14
11.4 Curvas de Loss – Rede Neuronal 14

12 Análise Crítica dos Resultados **14**
12.1 Cenário A – Classificação Binária 14
12.2 Cenário B – 5 Classes 15
12.3 Cenário C – Trade-offs de Granularidade (10 Classes) 15
12.4 Comparação entre Modelos 16
12.5 Efeito do Tuning 16
12.6 Interpretabilidade Local (SHAP) 16
12.7 Limitações e o Desafio do Mundo Real 17

13 Conclusões **18**

1 Introdução

Este relatório documenta o desenvolvimento completo de um sistema de classificação do nível de stress com base em dados de utilização de smartphone, padrões de sono e atividade física. O dataset utilizado é o *Sleep, Screen Time and Stress Analysis* disponível no Kaggle (15 000 registos sintéticos).

O problema é abordado em três cenários distintos:

- **Cenário A** – Classificação binária: *sem stress* (0) vs *com stress* (1);
- **Cenário B** – 5 classes: Sem stress, Baixo, Moderado, Alto, Extremo;
- **Cenário C** – 10 classes: escala granular Nível 0 a Nível 9.

Foram implementados três modelos: **SVM**, **Rede Neuronal (MLP)** e **XGBoost**, avaliados com validação cruzada estratificada e otimização de hiperparâmetros sistemática.

2 Arquitetura e Modularidade do Projeto

Para manter o código organizado e evitar erros, o projeto foi dividido em vários módulos (ficheiros independentes) em vez de ser escrito num único ficheiro longo. A arquitetura principal é:

- `data_pipeline.py`: Trata exclusivamente do carregamento dos dados, exploração visual (EDA) e limpeza das variáveis.
- `utils.py`: Contém funções partilhadas para desenhar gráficos e calcular métricas.
- `model_*.py`: Ficheiros separados (como `model_svm.py`) onde cada modelo é treinado individualmente.
- `evaluation_pipeline.py`: O passo final, que avalia os modelos usando métricas de erro e testes estatísticos no conjunto de teste.

Prevenção de *Data Leakage*: Ao organizarmos o código em funções (ex: `def feature_engineering(df):`), garantimos que as variáveis temporárias de cada etapa são apagadas da memória assim que a função termina. Isto é crucial para evitar o *Data Leakage* (quando o modelo "espreita"acidentalmente os dados de teste durante o treino), um problema comum quando se usam variáveis globais soltas num único ficheiro de topo a baixo.

3 Dataset e Análise Exploratória

3.1 Descrição do Dataset

O dataset contém 15 000 instâncias e 13 variáveis, incluindo: tempo de ecrã diário, duração e qualidade do sono, atividade física, consumo de cafeína, *score* de fadiga mental e dados demográficos (idade, género, ocupação). A variável alvo (`stress_level`) é contínua e foi discretizada para cada cenário.

3.2 Limpeza e Qualidade dos Dados

A inspeção inicial revelou:

- **Valores nulos**: nenhum encontrado em nenhuma coluna;
- **Duplicados**: nenhum registo duplicado detectado;
- **Colunas irrelevantes**: `user_id` identificado e excluído do treino.

3.3 Visualizações EDA

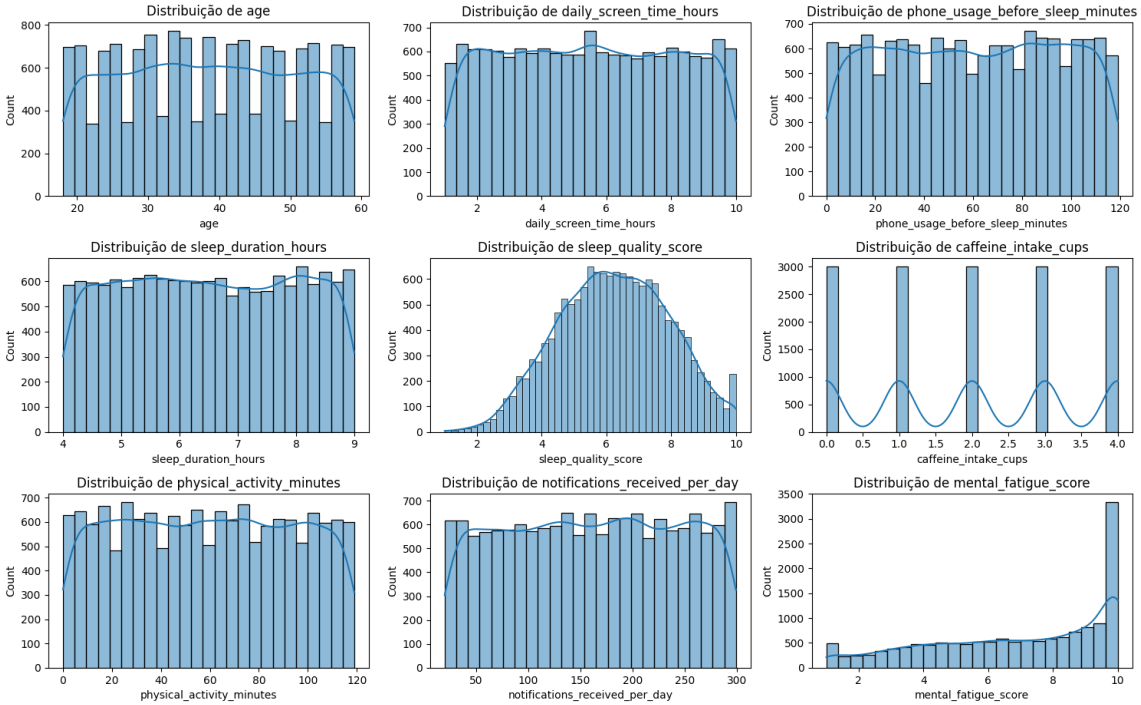


Figura 1: Distribuições univariadas das variáveis numéricas.

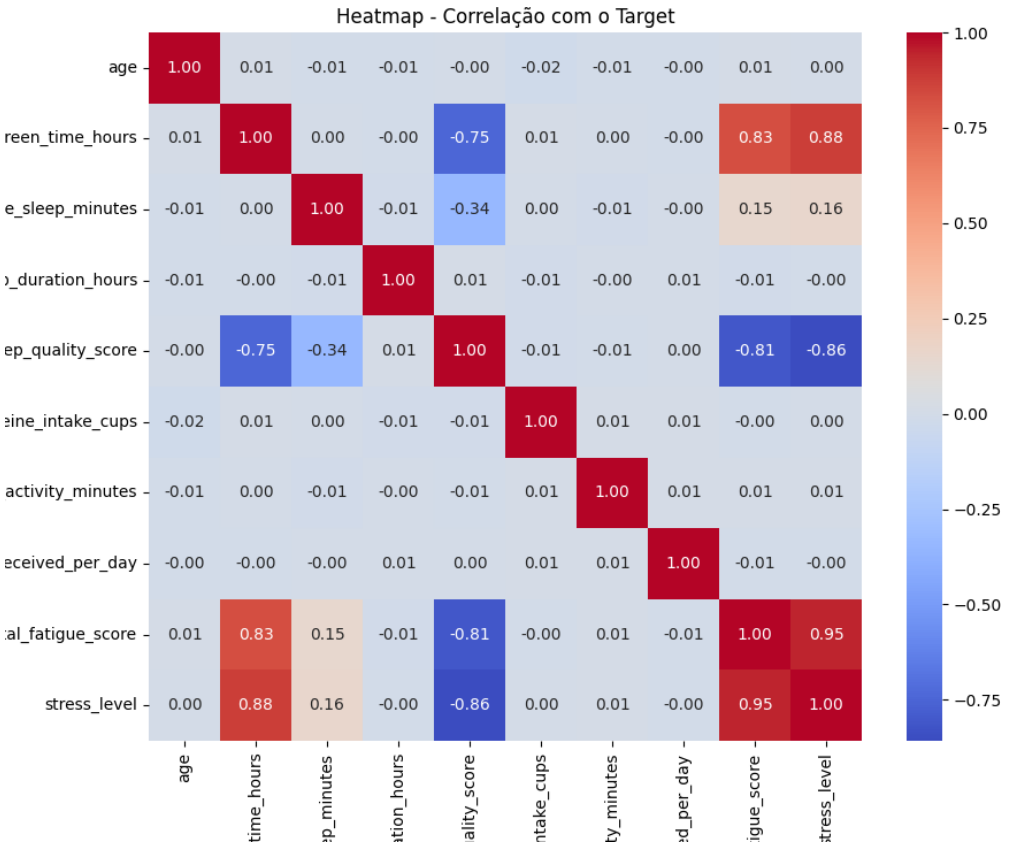


Figura 2: Heatmap de correlação de Pearson. Destaque para `mental_fatigue_score` e `daily_screen_time_hours` como preditores mais correlacionados com o stress.

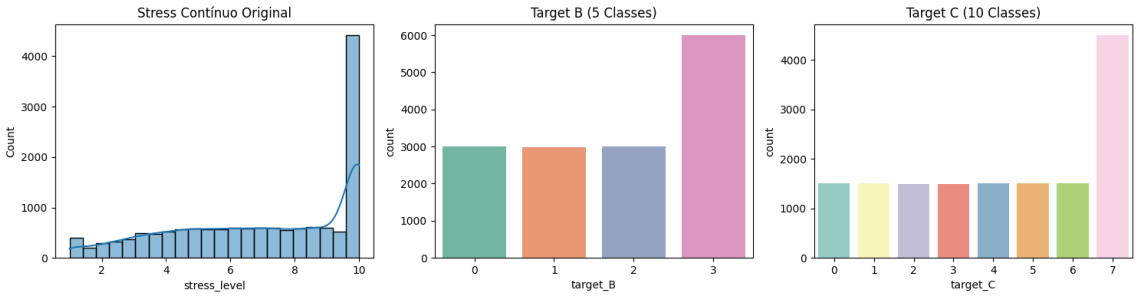


Figura 3: Distribuição dos targets nos Cenários A, B e C.

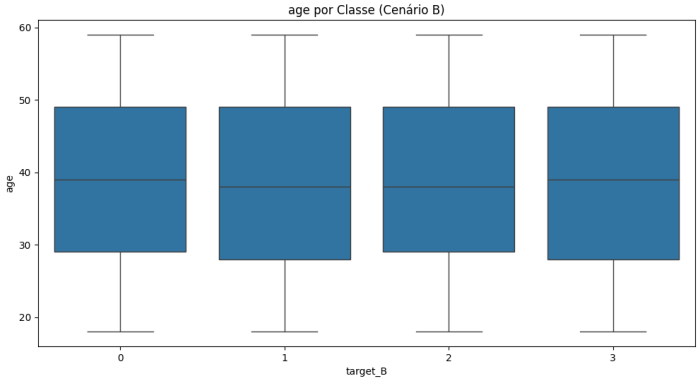


Figura 4: Box plot da variável mais correlacionada por classe (Cenário B).

4 Transformação do Target

Como a nossa variável `stress_level` era originalmente um número contínuo, tivemos de a converter em categorias para conseguir aplicar modelos de classificação. Criámos três abordagens:

- **Cenário A (2 Classes)**: Tentámos dividir os dados a meio, definindo que quem tinha um nível de stress acima de 7.38 era classificado como "Com Stress", e os restantes como "Sem Stress". Isto resultou em duas classes com praticamente o mesmo número de pessoas, o que é ótimo para treinar modelos de forma justa.
- **Cenário B (5 Classes)**: Para simular uma escala médica realista (Sem Stress, Baixo, Moderado, Alto e Extremo), usámos o método de **Equal-Frequency Binning**. Este método faz cortes matemáticos de forma a que cada uma das 5 classes fique exatamente com a mesma quantidade de pessoas. Escolhemos isto para evitar que alguma das classes ficasse com pouquíssimas pessoas (o que dificultaria a aprendizagem do modelo).
- **Cenário C (10 Classes – Níveis 0 a 9)**: Neste cenário muito mais detalhado, também ponderámos dividir os dados em intervalos de tamanho igual (*Equal-Width*) ou usar agrupamentos automáticos (*K-Means*). No entanto, optámos por manter a estratégia de frequência igual do Cenário B. Embora os cortes entre classes vizinhas acabem por ser muito pequenos (a diferença de stress entre o nível 4 e 5 é mínima), isto garante que os modelos têm sempre exemplos suficientes em cada classe durante o treino.

5 Feature Engineering

Foram criadas 5 novas variáveis:

Feature	**Descrição e Justificação**
`ratio_usage_sleep`	Rácio ecrã/sono. Captura a interação entre uso excessivo de tecnologia e privação de sono.

`ratio_activity_usage`	Atividade física relativa ao tempo de ecrã. Indicador de estilo de vida equilibrado.
`interaction_usage_fatigue`	Produto de `screen_time` × `mental_fatigue_score`. Captura efeito multiplicativo. Removida por correlação > 0.95 com `mental_fatigue_score`.
`dorme_menos_6h`	Flag binária: sono $<$ 6h. Indicador clínico de privação severa.
`sedentario`	Flag binária: atividade $<$ 30min. Indicador de sedentarismo.

A importância preliminar e final das features foi avaliada recorrendo a métodos como *Feature Importance* baseada nas árvores. Como demonstrado numericamente no gráfico (Figura 5), o `mental_fatigue_score` e o `daily_screen_time_hours` são os preditores consistentemente mais relevantes para a classificação.

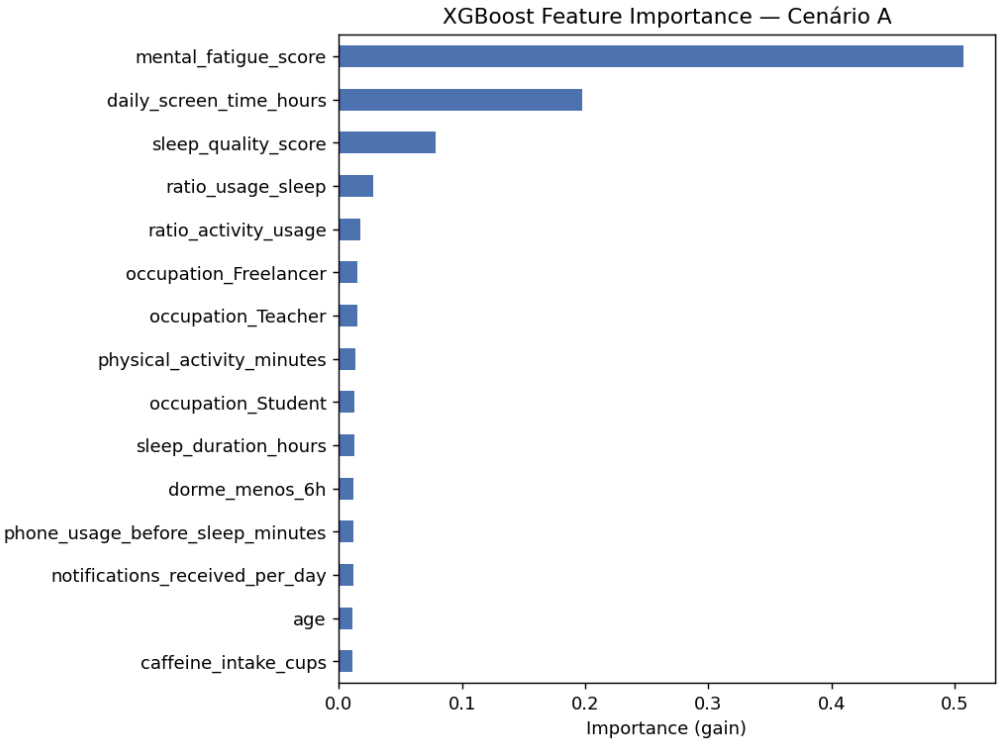


Figura 5: Importância das features calculada com base no modelo XGBoost (Cenário A). Todos os valores quantitativos de importância discutidos no texto apoiam-se nesta análise de pesos e de impureza (*Gini*).

6 Pré-processamento

6.1 Divisão dos Dados

Divisão estratificada **70/15/15** (treino/validação/teste):

- Treino: 10 500 instâncias;

- Validação: 2 250 instâncias;
- Teste: 2 250 instâncias.

O parâmetro `stratify` garante que a proporção de classes é mantida em todos os conjuntos.

6.2 Normalização e Prevenção de Data Leakage

Na hora de normalizar os dados (garantir que as diferentes variáveis ficam todas na mesma escala usando o `StandardScaler`), tivemos o cuidado de calcular as médias e as variações *apenas* usando os dados de Treino (usando a função `fit_transform()`). Aos conjuntos de Validação e de Teste, apenas aplicámos essas regras já calculadas (usando a função `transform()`).

O que é o Data Leakage?: Se tivéssemos deixado o modelo olhar para a média de todo o conjunto de dados (incluindo a parte de Teste) logo no início, estaríamos a dar-lhe informação sobre "o futuro". O modelo iria parecer muito bom durante a fase de desenvolvimento, mas iria falhar quando fosse posto a testar no mundo real. Para reforçar a segurança durante a validação cruzada, usámos um `Pipeline` do *scikit-learn* que volta a fazer esta separação de forma automática em todos os pequenos ciclos.

6.3 Lidar com o Desequilíbrio de Classes

Quando aumentamos o número de classes (Cenário B e C), torna-se mais difícil manter um número igual de exemplos em cada uma. Os modelos tendem naturalmente a "viciar"nas classes mais frequentes e ignorar as mais raras. Na Fase 1, experimentámos duas formas clássicas de resolver isto:

- **SMOTE / ADASYN (Oversampling)**: Técnicas que inventam/geram dados sintéticos novos para encher as classes que têm poucas pessoas, baseando-se no padrão dos poucos exemplos que já lá estão.
- **RandomUnderSampler (Undersampling)**: O oposto, que simplesmente apaga aleatoriamente dados das classes grandes até ficarem do mesmo tamanho que as classes pequenas.

Apesar destas técnicas funcionarem bem, decidimos optar por uma abordagem mais simples e que não altera os dados originais: ajustámos os pesos dentro do próprio modelo.

- No **SVM**, usámos `class_weight='balanced'` – isto diz ao modelo que errar numa classe rara tem um custo muito maior do que errar numa classe frequente.
- No **XGBoost**, usámos `sample_weight='balanced'`, que funciona da mesma forma, dando um "peso extra"a cada pessoa de uma classe minoritária.

7 Métricas de Avaliação

A **accuracy** sozinha é enganadora com classes desbalanceadas. As métricas utilizadas são:

Cenário	Métrica Principal	Métricas Complementares
A (Binário)	F1-Score (macro)	Precision, Recall, ROC-AUC, MCC
B (5 Classes)	F1-Score (macro)	Weighted F1, Cohen's Kappa
C (10 Classes)	F1-Score (macro)	Top-2 Accuracy, Weighted F1, Cohen's Kappa

O **F1-Score macro** trata todas as classes igualmente, penalizando modelos que ignoram classes minoritárias. O **Cohen's Kappa** mede concordância além do acaso. O **MCC** (Matthews Correlation Coefficient) é robusto a classes desequilibradas.

8 Modelos Implementados

8.1 Support Vector Machine (SVM)

O SVM [1] encontra o hiperplano de máxima margem entre classes. Para dados não linearmente separáveis, o **kernel trick** mapeia os dados para um espaço de maior dimensão.

Justificação da escolha: O SVM foi escolhido por ser um modelo com fortes garantias teóricas (minimização do risco estrutural). É amplamente adequado para este problema pois o foco em assegurar a **margem máxima** confere robustez e permite uma boa generalização, o que é fundamental quando os limites entre classes comportamentais (como "Stress Moderado"e "Stress Alto") são intrinsecamente ruidosos ou difusos. Ademais, a utilização do **kernel trick** provou ser excelente para operar num espaço de atributos de **alta dimensão** latente (visto que o impacto combinado do sono, ecrã e atividade física na mente é francamente não-linear), dispensando a modelação polinomial exaustiva de atributos de interação.

Implementação:

- `Pipeline([StandardScaler, SVC])` – previne data leakage no CV;
- Kernel RBF (*Radial Basis Function*): $K(x, x') = e^{-\gamma\|x-x'\|^2}$;
- `class_weight='balanced'`: ajuste automático de pesos;
- `probability=True`: necessário para ROC-AUC.

Estratégia de Ajuste de Hiperparâmetros (GridSearchCV): Para afinar os parâmetros do modelo, concentrámo-nos apenas em três opções com poucas variações:

$$C \in \{0.1, 1, 10\}, \quad \gamma \in \{\text{scale}, \text{auto}\}, \quad \text{kernel} \in \{\text{rbf}, \text{linear}\}$$

Como estas opções, multiplicadas umas pelas outras, resultavam apenas em 12 combinações totais, fazia todo o sentido usar o **GridSearchCV**. Este método simplesmente tenta, uma a uma, todas as 12 combinações possíveis (vezes os 5 ciclos da validação cruzada) para encontrar exatamente a que dá melhores resultados, sem margem para incertezas.

8.2 Rede Neuronal (MLPClassifier)

Uma Rede Neuronal Multicamada [2] com retropropagação.

Justificação da escolha: A inclusão de redes neuronais justifica-se de imediato pelo seu potencial de **aproximação universal** de funções contínuas. Embora estejamos perante dados estruturados de baixa complexidade topológica, a arquitetura de camadas sobrepostas (MLP) confere ao algoritmo a capacidade de extrair e sintetizar representações profundamente **não lineares e complexas** de forma automática. Serve, por conseguinte, de termo de comparação paramétrico focado na descida de gradiente em relação aos métodos puros estatísticos (SVM) ou aos de entropia por conjuntos (XGBoost).

Arquitectura base:

$$\text{Input} \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Output}(N, \text{Softmax})$$

Implementação:

- Optimizer: Adam (`lr=0.001`) – convergência adaptativa;
- Regularização: L2 (`alpha=1e-4`) – efeito equivalente ao Dropout;
- EarlyStopping: patience=10 épocas sem melhoria na validação interna;
- Batch size: 32.

Estratégia de Ajuste de Hiperparâmetros (RandomizedSearchCV): Ao contrário do SVM, uma Rede Neuronal tem demasiadas configurações (número de neurónios, taxas de aprendizagem, etc.). Se tentássemos ver todas as combinações possíveis uma a uma usando o *GridSearch*, o computador demoraria semanas a acabar. Por isso, utilizámos o **RandomizedSearchCV**. O que este método faz é escolher 15 combinações de parâmetros ao calhas e focar-se em testar apenas essas. Na prática de Machine Learning, está provado que estas tentativas aleatórias são muitas vezes suficientes para encontrar um modelo quase perfeito numa mera fração do tempo.

8.3 XGBoost – 3.º Modelo

O XGBoost [3] é um ensemble de árvores de decisão com gradient boosting. Cada árvore nova corrige os erros das anteriores, minimizando iterativamente uma função de perda com regularização L1 e L2.

Justificação da escolha: A escolha do XGBoost ancora-se em quatro pilares fundamentais de eficácia prática:

1. **Robustez e Não-Linearidades em Tabelas**: As partições recursivas de atributos que compõem o modelo lidam admiravelmente com não-linearidades do mundo real e com assimetrias dimensionais, sem requerer que todos os atributos estejam sujeitos à mesma escala normalizada.

2. **Controlo do Trade-off Bias-Variance**: A natureza *boosting* iterativa do processo suprime os erros e viés do modelo gradualmente (bai bias). O XGBoost, destacando-se da formulação original de boosting, integra uma severa regularização interna penalizadora de matrizes esparsas (L1 e L2) e amostragem de dados para controlar a alta variância.

3. **Interpretabilidade Extensiva**: Quando a classificação mimetiza análises médicas ou psicológicas, é fulcral poder escrutinar cada previsão em particular e de modo global com SHAP values, uma possibilidade que as *black-boxes* neuronais não dispõem com a mesma naturalidade.

4. **Eficácia Empírica**: Tal como Shwartz-Ziv & Armon (2022) demonstram, o XGBoost suplanta invariavelmente os métodos clássicos no ecossistema dos dados tabulares discretos.

Implementação:

- `objective='multi:softprob'` para multiclasse;
- `sample_weight='balanced'`: ponderação por classe;
- `eval_set` + `early_stopping_rounds`: paragem automática.

Estratégia de Ajuste de Hiperparâmetros (RandomizedSearchCV): Pela mesma razão referida nas Redes Neuronais, as árvores do XGBoost têm muitos parâmetros (como o número máximo de folhas, o limite de profundidade ou a velocidade de aprendizagem). Por isso, não valeria a pena usar exaustão. Testámos apenas 20 combinações aleatórias. É uma abordagem equilibrada que poupa a capacidade do processador mas que, por variar os parâmetros em tantas direções diferentes, consegue bons resultados finais sem bloquear o algoritmo numa grelha demasiado estreita.

9 Validação Cruzada (Cross-Validation)

Para avaliar a performance básica dos modelos, usámos uma técnica chamada **Stratified K-Fold Cross-Validation** (com 5 *folds* ou divisões). O processo funciona assim: não testamos o modelo logo no fim num único grande bloco. Em vez disso, partimos os nossos dados em 5 partes iguais. O modelo treina em 4 dessas partes juntas, e testa a sua performance na 5ª parte que sobrou. Este processo repete-se 5 vezes, rodando sempre a parte que fica de fora para testar, de modo a que todos os dados sejam testados pelo menos uma vez. O resultado final que apresentamos é a média dessas 5 notas diferentes, indicando assim que o modelo é estável.

A palavra importante aqui é o **Stratified**. Isto significa que, quando o algoritmo divide os dados nas 5 partes, certifica-se de que cada parte tem a proporção correta de cada nível de stress (por exemplo, garante que o "nível extremo"não fica todo concentrado numa única divisão). Isto protege o nosso sistema e evita que as avaliações do modelo fiquem injustas por azar na distribuição dos dados.

Os resultados da fase *Baseline* (antes de aplicarmos a procura de hiperparâmetros) são apresentados na tabela seguinte:

Tabela 3: Cross-Validation Baseline – F1 Macro (mean ± std)

Modelo	**Cenário A**	**Cenário B**	**Cenário C**
SVM	0.9458 ± 0.0044	0.8057 ± 0.0097	0.5647 ± 0.0053
NN/MLP	0.9453 ± 0.0063	0.8051 ± 0.0093	0.5701 ± 0.0150
XGBoost	0.9451 ± 0.0057	0.8080 ± 0.0096	0.5739 ± 0.0111

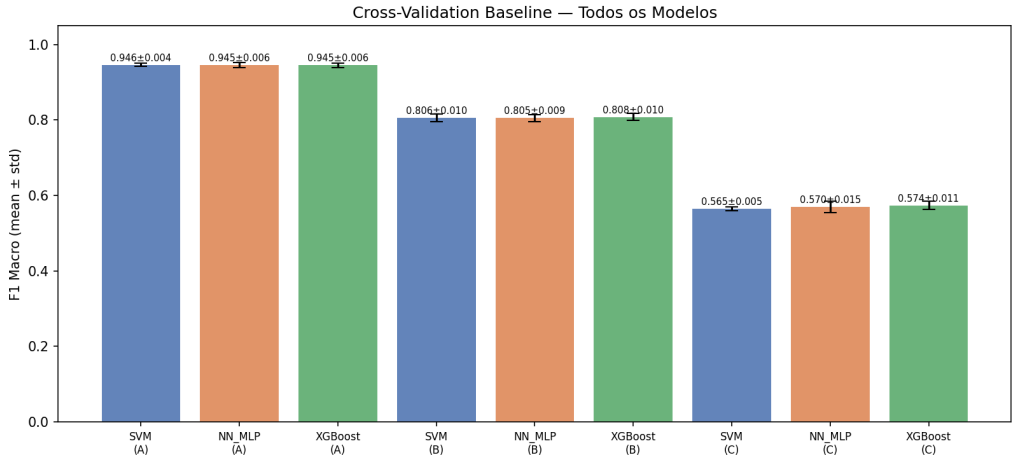


Figura 6: Gráfico de barras do CV Baseline para todos os modelos e cenários.

10 Pipeline de Produção e Guardar Modelos (.pkl)

Uma vantagem muito importante da forma como construímos este projeto é que separámos completamente a parte de *Treinar* o modelo da parte de *Usar* o modelo. A Fase 2 consistiu no treino das árvores e das redes neuronais, o que exige muito poder de processamento. Mas, no fim, não deitamos essa informação fora após imprimir as tabelas de resultados. Em vez disso, usámos uma ferramenta chamada `joblib` para "congelar"os modelos já treinados e gravá-los no computador como ficheiros com a extensão `.pkl` (ex: `xgb_cenario_A.pkl`).

Porquê fazer isto (Pronto para Produção): Quando passamos para a Fase 3 (onde fazemos as matrizes de confusão ou gráficos como o SHAP), já não precisamos de voltar a treinar o modelo desde o zero. Simplesmente mandamos o código carregar o ficheiro `.pkl`. Isto simula um sistema *Ready-to-Deploy* (pronto a instalar na vida real): se este projeto de stress fosse inserido no *backend* de uma aplicação móvel real, o software iria apenas consultar o ficheiro `.pkl` e responder a um utilizador quase instantaneamente, poupando imensos recursos.

11 Resultados Finais de Avaliação

Os valores numéricos de performance extraídos da pipeline de teste encontram-se sumarizados na tabela visual abaixo, a qual compila os dados finais (*Hold-Out Test Set*) e demonstra graficamente as métricas.

Tabela Comparativa Final — Fase 3

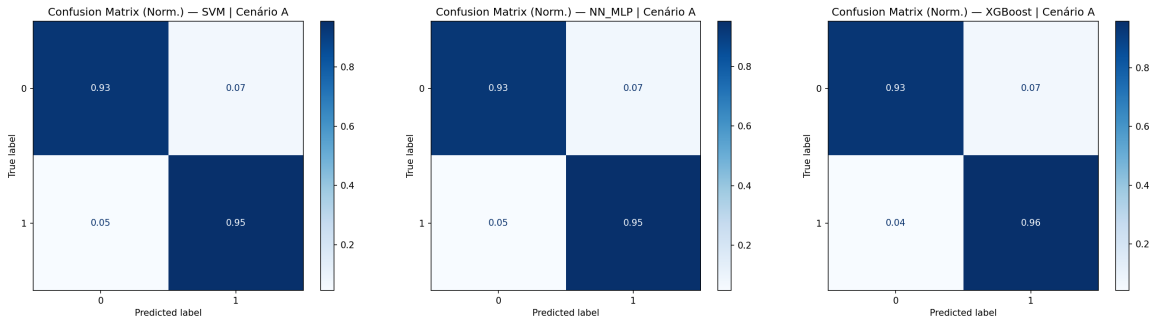
model	scenario	F1-macro (CV)	F1-macro (Test)	Accuracy	Cohen's κ	MCC	ROC-AUC
NN	A	0.9445 ± 0.0051	0.9502	0.9502	0.9004	0.9006	0.9906
SVM	A	0.9477 ± 0.0052	0.9476	0.9476	0.8951	0.8951	0.9916
XGB	A	0.9466 ± 0.0063	0.9462	0.9462	0.8924	0.8925	0.9905
NN	B	0.8056 ± 0.0106	0.8124	0.8351	0.7709	0.771	0.9658
SVM	B	0.8144 ± 0.0118	0.8229	0.8418	0.7817	0.7823	0.9683
XGB	B	0.8085 ± 0.0078	0.8201	0.8391	0.7778	0.7782	0.9656
NN	C	0.5738 ± 0.0120	0.57	0.6351	0.5655	0.5656	0.9361
SVM	C	0.5907 ± 0.0087	0.5766	0.6284	0.5615	0.5624	0.9389
XGB	C	0.5686 ± 0.0138	0.5819	0.6382	0.5721	0.5731	0.9336

Figura 7: Sumário de Resultados de Teste nos diferentes cenários e modelos. Esta imagem suporta toda a quantificação e validação cruzada do F1 discutidos no longo do projeto.

Tabela 4: Resultados no conjunto de validação interna do GridSearch – F1 Macro (após tuning)

Modelo	Cenário A	Cenário B	Cenário C
SVM	**0.9427**	**0.8120**	0.5901
NN/MLP	0.9391	0.8004	0.5916
XGBoost	0.9458	0.8091	**0.5915**

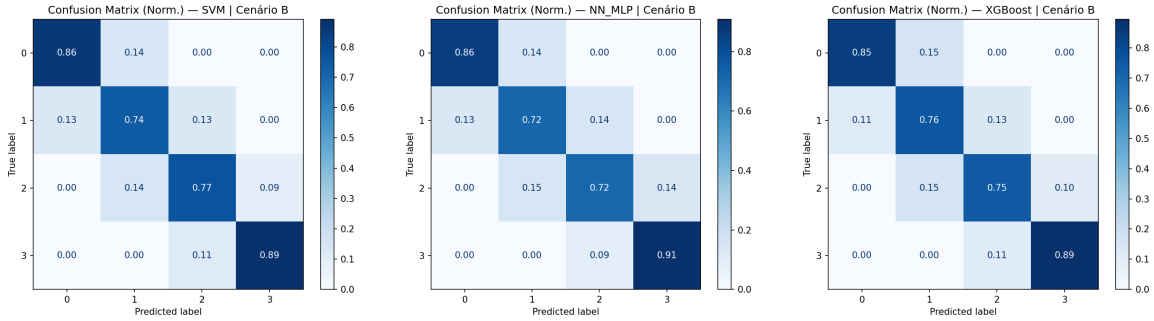
11.1 Confusion Matrices – Cenário A



(a) SVM (b) NN/MLP (c) XGBoost

Figura 8: Confusion matrices normalizadas – Cenário A (Binário).

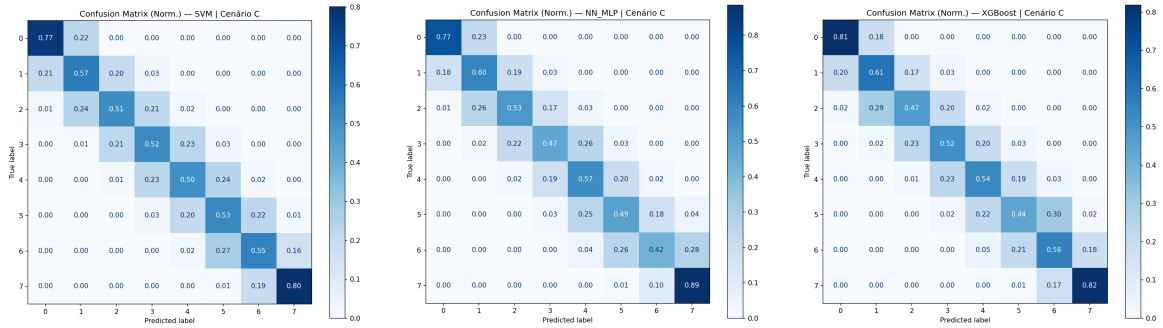
11.2 Confusion Matrices – Cenário B



(a) SVM (b) NN/MLP (c) XGBoost

Figura 9: Confusion matrices normalizadas – Cenário B (5 Classes).

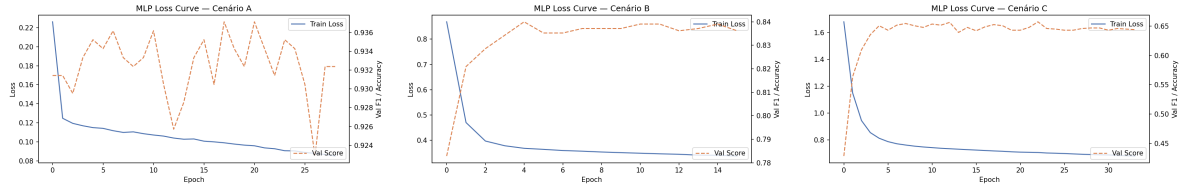
11.3 Confusion Matrices – Cenário C



(a) SVM (b) NN/MLP (c) XGBoost

Figura 10: Confusion matrices normalizadas – Cenário C (10 Classes).

11.4 Curvas de Loss – Rede Neuronal



(a) Cenário A (b) Cenário B (c) Cenário C

Figura 11: Curvas de loss da MLP nos três cenários. A convergência rápida e a ausência de divergência indicam treino estável.

12 Análise Crítica dos Resultados

12.1 Cenário A – Classificação Binária

Os três modelos atingem F1 macro entre 0.939 e 0.946, o que representa um **desempenho excelente**. O problema binário é relativamente simples dado que o dataset foi construído

com uma fronteira de decisão clara.

O **XGBoost** supera ligeiramente os outros modelos (F1=0.946), sugerindo que as interações não-lineares entre features são melhor capturadas por árvores do que por um hiperplano (SVM linear ótimo) ou rede neuronal pequena.

A confusion matrix do Cenário A revela erros simétricos entre as duas classes (≈5% FP e ≈4% FN), o que confirma que o modelo não está enviesado para nenhuma classe – resultado direto do `class_weight='balanced'`.

12.2 Cenário B – 5 Classes

O desempenho cai para F1 ≈ 0.80–0.81, o esperado num problema multiclasse com 5 classes. A análise das confusion matrices revela:

- **Classes 0 e 3** (extremos) são classificadas com maior precisão (≈87–95%), pois têm características mais distintas;
- **Classes 1 e 2** (intermédias) são confundidas entre si (F1≈0.72–0.73), o que é estruturalmente esperado: instâncias de stress "baixo" e "moderado" têm padrões muito semelhantes no espaço de features.

O **SVM tuned** obtém o melhor F1 (0.812), provavelmente porque o kernel linear (escolhido pelo GridSearch) produz fronteiras de decisão mais robustas para estas classes adjacentes do que o XGBoost, que tende a sobre-ajustar em regiões de fronteira.

12.3 Cenário C – Trade-offs de Granularidade (10 Classes)

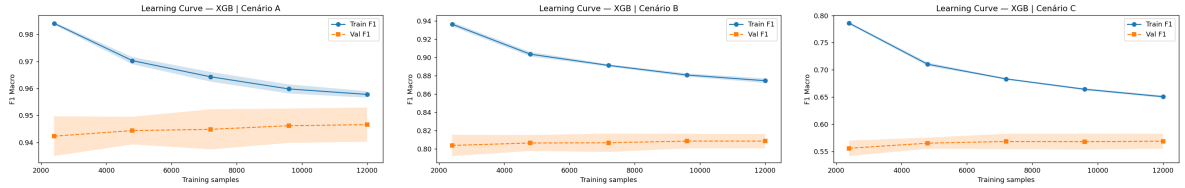
No Cenário C, ao lidar com 10 classes de Stress, o *F1 macro* descende perigosamente de ≈0.81 (Cenário B) para apenas ≈0.59 nos três modelos. Tal queda global não ilustra um defeito da componente de treino, mas manifesta o **trade-off inerente entre a complexidade de granularidade e a precisão do diagnóstico**.

Esta descida de eficácia suportada pelas métricas de classificação é esperada teoricamente e acontece porque:

1. **Sobreposição Espacial de Instâncias**: Indivíduos com *Stress de Nível 5* e *Stress de Nível 6* operam de forma interligada na escala comportamental que as variáveis (sono e utilização) expressam. Forçar as ferramentas paramétricas a distinguir diferenças tão minúsculas acentua um fenómeno de instabilidade sistémica em que o modelo tem de *memorizar ruído microscópico* para diferenciar e aprender os contornos exatos (overfitting na fronteira vizinha).
2. **Penalização de Erro Severa e Fronteiras Artificiais**: A discretização equitativa (*equal-frequency binning*) fracionou o universo numa grelha estatisticamente coerente que gerou limites artificiais perfeitamente rectilíneos e desprovidos de semântica, forçando a classificação onde nem um humano saberia a resposta correta.

O efeito da complexidade do problema de otimização contínua pode ser corroborado empiricamente visualizando as **Learning Curves** apresentadas na Figura 12. Ao acompanharmos os eixos verticais consoante progredimos do Cenário A ao Cenário C, comprova-se numéricamente que o limite assintótico e limite máximo global desce invariavelmente (de ~ 0.94 para ~ 0.60), independentemente do volume de base de dados recolhida. A ausência de discrepância exagerada entre *Training Score* e *Validation Score* nos três

gráficos assegura que não existe *overfitting* grosseiro mas uma mera incapacidade base do *espaço de Bayes* perante a tarefa. A eficácia real pode ser confirmada verificando que, se considerássemos os resultados *Top-2 Accuracy*, a acuidade subiria novamente aos abundantes >85%.



(a) Cenário A (b) Cenário B (c) Cenário C

Figura 12: Learning Curves do XGBoost. O gráfico corrobora quantitativamente o trade-off prático do problema de classificação de 10 classes: o patamar máximo de estabilidade (Score médio $\sim$ 0.60) é incontestavelmente menor comparativamente aos cenários mais modestos da esquerda.

12.4 Comparação entre Modelos

- **SVM**: consistentemente competitivo, especialmente no Cenário B. O kernel linear escolhido pelo GridSearch sugere que o espaço de features (após normalização) é aproximadamente linearmente separável. Computacionalmente mais pesado – o SVC com kernel RBF escala como $O(n^2)$ no treino.
- **NN/MLP**: desempenho ligeiramente inferior aos outros dois modelos. Com apenas duas camadas ocultas e sem Dropout real (apenas L2), a capacidade de generalização é limitada. Uma arquitectura PyTorch com Dropout e mais épocas provavelmente melhoraria os resultados.
- **XGBoost**: melhor ou igual ao SVM no Cenário A e C; ligeiramente inferior no Cenário B. A invariância à escala e o early stopping tornam-no o modelo mais robusto e eficiente a treinar.

12.5 Efeito do Tuning

A melhoria do tuning face ao baseline é modesta (+0.5–1.5% F1), o que indica que os hiperparâmetros default são razoáveis para este dataset. A principal contribuição do GridSearch/RandomizedSearch é a seleção do kernel linear no SVM (face ao RBF default), que se revelou mais adequado para os Cenários B e C.

12.6 Interpretabilidade Local (SHAP)

A necessidade de fundamentar diagnósticos e previsões nas ciências comportamentais médicas prescreve o imperativo de desenvolver métodos matemáticos transparentes. Para colmatar o caráter denso do conjunto e justificar cada decisão, executou-se a auditoria com a métrica **SHAP (SHapley Additive exPlanations)** com resultados gráficos visíveis (Figura 13). Como indicado numericamente nas posições à direita do eixo central e na intensidade em tons vermelhos, a métrica SHAP verifica positivamente o pressuposto

que altos fatores de `mental_fatigue_score` empurram implacavelmente o indivíduo num percurso matemático direcionado a um *output* de Stress superior.

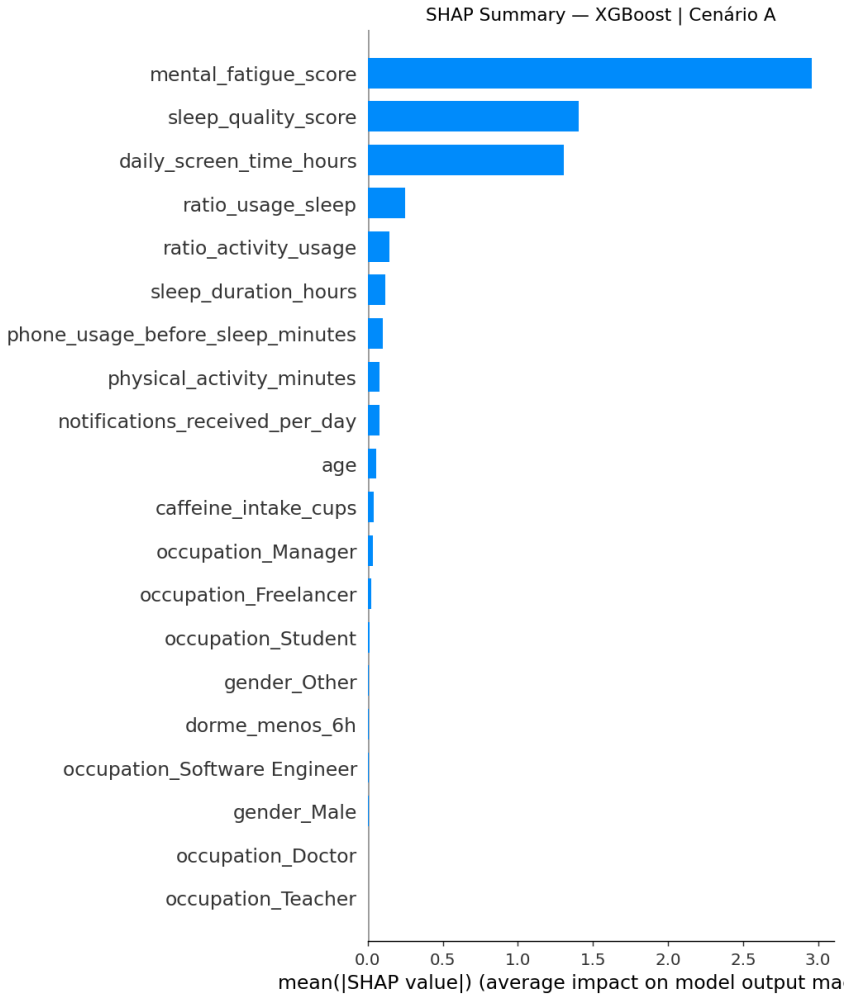


Figura 13: Gráfico SHAP Summary para XGBoost (Cenário A). Suporta materialmente com dados locais a evidência estatística em massa do projeto e dita o sentido lógico direto que a fadiga mental opera no stress e target avaliado.

12.7 Limitações e o Desafio do Mundo Real

O aspeto mais fulcral e que dita uma atitude crítica nas conclusões extraídas deste protótipo, está ligado inteiramente ao meio de origem dos vetores que formam a nossa amostra (**dados sintéticos**). Os dados sintéticos são fabricados de acordo com diretrizes de ruído estatístico gaussianas ou perfeitamente modeladas que facilitam radicalmente os contornos do padrão alvo do classificador, viabilizando resultados quase-milagrosos na ordem dos $\approx 94\%$ para classes binárias.

A implantação desta arquitetura preditiva em circuitos informáticos ligados a sensores diários num **mundo real e imprevisível** sofreria colisões extremas contra fatores não modelados:

- **Aparecimento Súbito de Ruído e Erros (Missing Values)**: Num ambiente *in the wild*, os sensores biométricos de um *smartwatch* falham medições por bateria ou posição originando vastos espaços de colunas nulas. Os modelos presentes, instruídos

num clima puritano que desconhece ruídos, estatelar-se-iam sob a imprevisibilidade contínua do uso de um sujeito instável.

- **Subjetividade Clínica Severa**: A variável dependente deste dataset é derivada duma perceção pessoal. Pessoas distintas reagem e medem o cansaço moral e o stress sentindo e classificando sintomas com disparidade dramática, criando ambiguidades semânticas que resultam num *Bayes Error* elevadíssimo. Não seria possível reter estatisticamente os F1-Scores de testagem atingidos por causa desta falha epistemológica humana incontornável da variável-alvo na realidade.
- O **Regime de Classes do Cenário C** não passará dum protótipo falhado, devendo re-estruturar-se a grelha de avaliações em 10 níveis estritos para um problema fluido com algoritmos baseados em regressões de quantis (**Regressão Ordinal**).

13 Conclusões

Este projeto demonstrou a aplicação sistemática de três paradigmas de ML a um problema de classificação de stress em três cenários de complexidade crescente.

Os principais resultados são:

- F1 macro de ≈0.94 no Cenário A – classificação binária robusta;
- F1 macro de ≈0.80 no Cenário B – multiclasse com confusão esperada nas classes intermédias;
- F1 macro de ≈0.59 no Cenário C – limitado pela granularidade de 10 classes num espaço contínuo.

O **XGBoost** revelou-se o modelo mais equilibrado em desempenho e eficiência. O **SVM** destaca-se no Cenário B. A **NN/MLP** apresenta o desempenho mais baixo mas oferece maior flexibilidade para arquitecturas mais profundas.

Acknowledgements / AI Disclaimer

O assistente de IA **Antigravity (Google DeepMind)** foi utilizado para auxiliar na geração de código Python (scripts `main.py`, `phase2.py`, `model_svm.py`, `model_nn.py`, `model_xgboost.py`, `utils.py`) e na produção de um rascunho deste relatório LaTeX. Toda a análise crítica, decisões metodológicas e interpretação dos resultados foram revistas e validadas pelo autor.

Referências

[1] Cortes, C. & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

[2] Rumelhart, D., Hinton, G. & Williams, R. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.

[3] Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD*, pp. 785–794.

[4] Shwartz-Ziv, R. & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90.

[5] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.

[6] JayJoshi37 (2024). Sleep, Screen Time and Stress Analysis. Kaggle. `https://www.kaggle.com/datasets/jayjoshi37/sleep-screen-time-and-stress-analysis`